Модуль 1 Практика 3

Раздел 1

1) Войти под пользователем user1 из практики 2

```
root@eltex-practice2-pg1-v17:~# su - user1
Warning: your password will expire in 5 days.
user1@eltex-practice2-pg1-v17:~$
```

*Скрин 1) Вход под пользователем **user1**.*

2) Подсчитать количество процессов, имеющих несколько потоков выполнения
ps -eo nlwp= | awk '$1 > 1 { count++ } END { print count+0 }' - подсчитать процессы (знак = в конце убирает заголовок).

```
user1@eltex-practice2-pg1-v17:~$ ps -eo nlwp= | awk '$1 > 1 { count++ } END { print count+0 }'
10
```

Скрин 2) Подсчет процессов с количеством потоков >1.

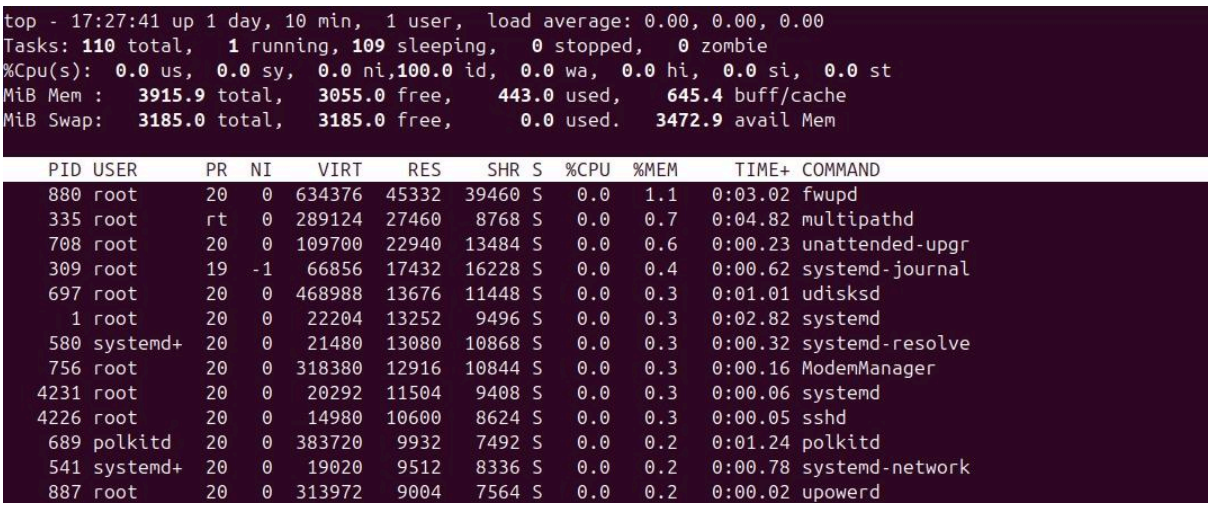
3) Запустить top и настроить вывод полей с информацией о процессе следующим образом:

- **отсортировать список процессов по убыванию значения поля VIRT, повторить для полей RES и SHR;**
- **удалить поля VIRT, RES, SHR и добавить поле RUSER, сделать так, чтобы это поле было показано после поля USER;**

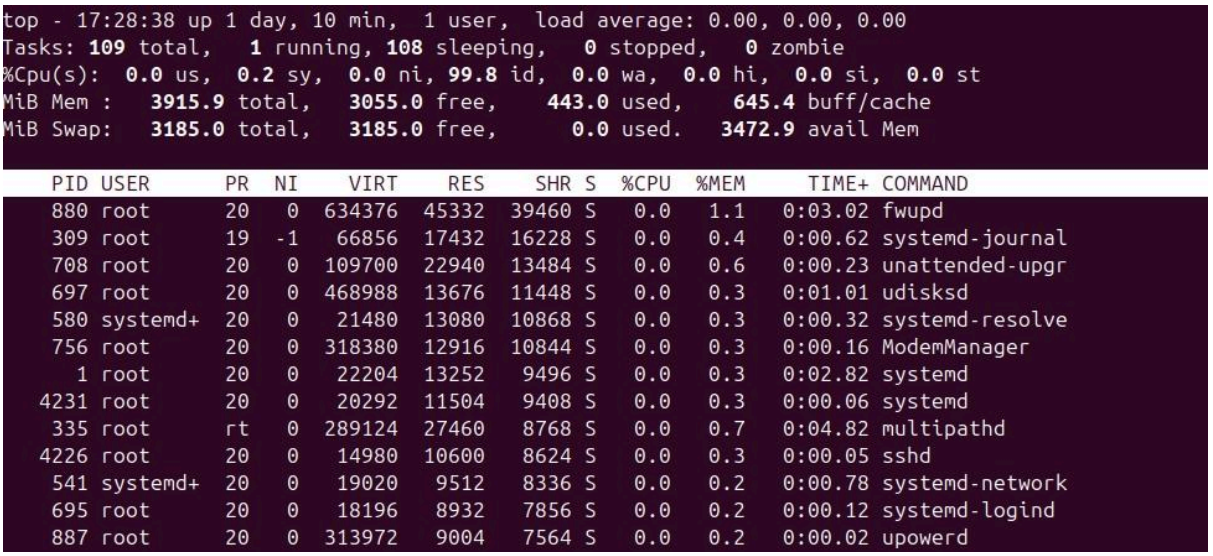
```
top - 17:10:25 up 23:52,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 110 total,   2 running, 108 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   3057.8 free,    440.4 used,    645.2 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3475.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    880 root      20   0  634376  45332  39460 S   0.0   1.1   0:02.99 fwupd
    697 root      20   0  468988  13676  11448 S   0.0   0.3   0:01.00 udisksd
    689 polkitd   20   0  383720   9932   7492 S   0.0   0.2   0:01.23 polkitd
    756 root      20   0  318380  12916  10844 S   0.0   0.3   0:00.16 ModemManager
    887 root      20   0  313972   9004   7564 S   0.0   0.2   0:00.02 upowerd
    335 root      rt   0  289124  27460   8768 S   0.0   0.7   0:04.76 multipathd
    715 syslog    20   0  222516   6208   4524 S   0.0   0.2   0:00.10 rsyslogd
    708 root      20   0  109700  22940  13484 S   0.0   0.6   0:00.23 unattended-upgr
    582 systemd+  20   0   91036   7892   6916 S   0.0   0.2   0:00.32 systemd-timesyn
    690 root      20   0   80716   4308   4008 S   0.0   0.1   0:33.66 qemu-ga
    309 root      19  -1   66856  17420  16216 S   0.0   0.4   0:00.61 systemd-journal
    374 root      20   0   29336   8000   4996 S   0.0   0.2   0:00.35 systemd-udevd
      1 root      20   0   22204  13252   9496 S   0.0   0.3   0:02.81 systemd
```

*Скрин 3) Список процессов, отсортированный по полю **VIRT**.*

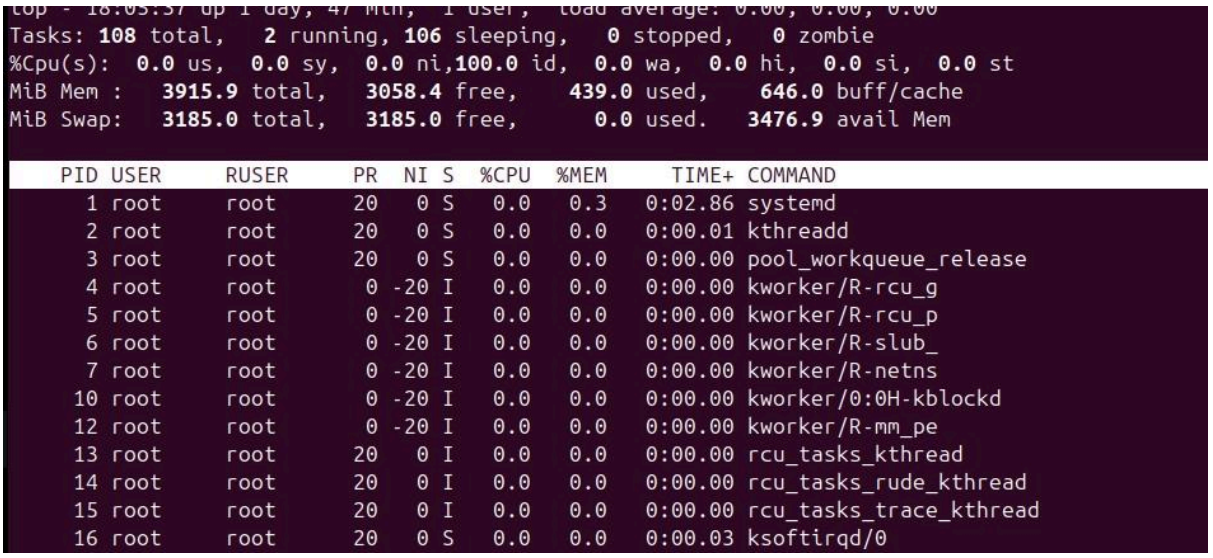


*Скрин 4) Список процессов, отсортированный по полю **RES**.*



*Скрин 5) Список процессов, отсортированный по полю **SHR**.*

VIRT отражает общий объем виртуальной памяти, который процесс зарезервировал у операционной системы.
RES отражает объем физической памяти (RAM) без учета swap, которую процесс занимает в данный момент, включая разделяемые библиотеки (***SHR***).
SHR *отражает часть от показателя RES, которая показывает, сколько физической памяти процесса является общей с другими процессами (общие библиотеки).*

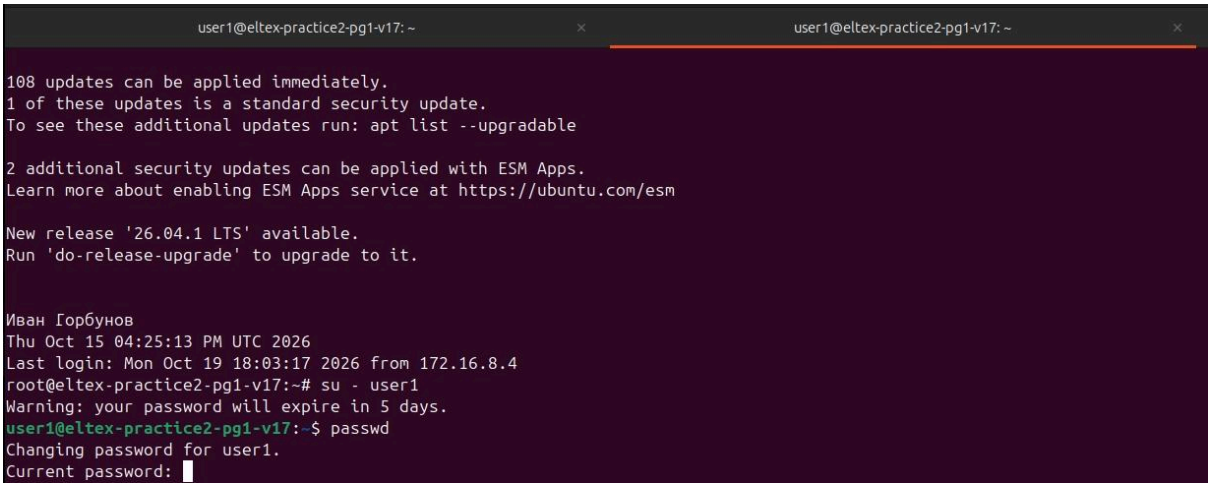


*Скрин 6) Список процессов без **VIRT, RES, SHR** и добавленным **RUSER**.*

USER - эффективный пользователь процесса. Его идентификатор используется при большинстве проверок прав доступа.

RUSER - реальный пользователь процесса, то есть идентификатор, связанный с тем, кто его запустил.
Например, для программы с **SUID**, принадлежащей root, при запуске пользователем user1, **USER** будет - root, а **RUSER** - user1, к примеру утилита **passwd**.

4) В другом терминальном окне выполнить команду *passwd* и оставить ее в состоянии запроса текущего пароля

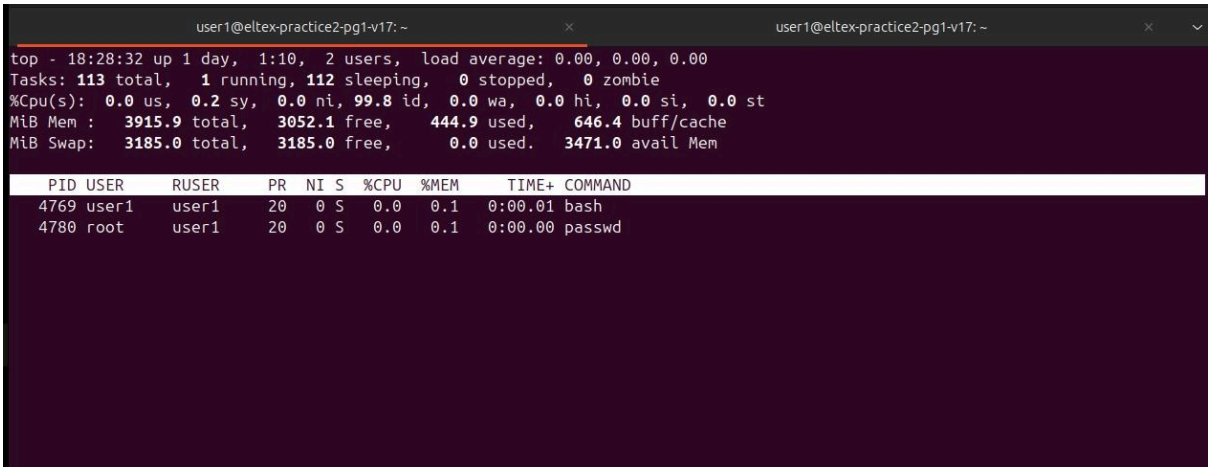


*Скрин 7) Утилита **passwd** в состоянии запроса текущего пароля в новом терминальном окне.*

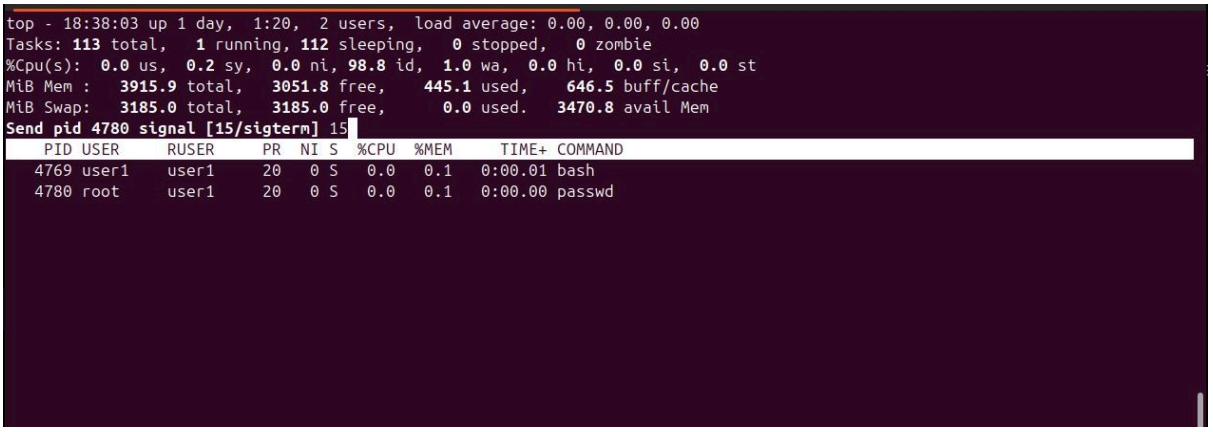
5) Перейти в терминальное окно с *top* и выполнить следующие действия:

- **вывести все процессы, для которых реальным пользователем является пользователь, которым вошли в сеанс;**
- **найти процесс, запущенный командой *passwd*;**

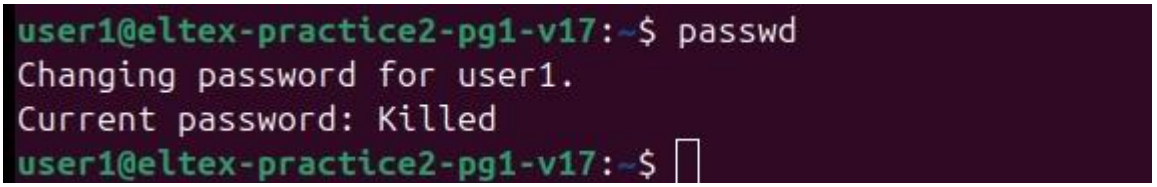
- **отправить этому процессу сигналы *SIGTERM*, *SIGINT*, *SIGQUIT*, *SIGKILL* используя их числовые идентификаторы**



*Скрин 8) Процессы для которых **RUSER=user1**, включая процесс, запущенный командой **passwd**.*

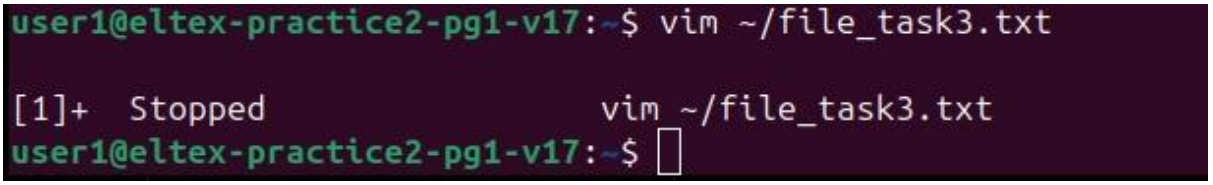


*Скрин 9) Отправка сигналов процессу **passwd (15, 2, 3, 9)**.*



Скрин 10) Процесс завершен.

6) Выполнить команду vim ~/file_task3.txt и нажать *Ctrl-Z*



*Скрин 11) Нажатие **Ctrl-Z** после запуска редактора.*

7) Выполнить команду sleep 600, нажать *Ctrl-Z* и выполнить команду *jobs*

```
user1@eltex-practice2-pg1-v17:~$ sleep 600
^Z
[2]+  Stopped                 sleep 600
user1@eltex-practice2-pg1-v17:~$ jobs
[1]-  Stopped                 vim ~/file_task3.txt
[2]+  Stopped                 sleep 600
user1@eltex-practice2-pg1-v17:~$
```

*Скрин 12) Выполнение команды **sleep 600**, **Ctrl-z**, **jobs**.*

8) Последнее задание (sleep 600) сделать фоновым

```
user1@eltex-practice2-pg1-v17:~$ bg %2
[2]+ sleep 600 &
user1@eltex-practice2-pg1-v17:~$ jobs
[1]+  Stopped                 vim ~/file_task3.txt
[2]-  Running                 sleep 600 &
```

Скрин 13) Установка выполнять последнее задание в фоне.

9) Изменить число NICE у задания (sleep 600), сделав его равным 10

```
user1@eltex-practice2-pg1-v17:~$ jobs -l
[1]+  4859 Stopped                 vim ~/file_task3.txt
[2]-  4872 Running                 sleep 600 &
user1@eltex-practice2-pg1-v17:~$ renice 10 -p 4872
4872 (process ID) old priority 0, new priority 10
```

*Скрин 14) Изменение числа **NICE**.*

10) Проверить, что число NICE у этого задания изменилось

```
user1@eltex-practice2-pg1-v17:~$ ps -o pid,ni,comm -p 4904
    PID  NI COMMAND
   4904  10 sleep
```

*Скрин 15) Проверка **NICE** (**PID** на скриншоте отличается от скриншота 14, т.к. процесс был перезапущен по истечению времени).*

11) Сделать задание vim ~/file_ task3.txt активным и выйти из редактора

```
user1@eltex-practice2-pg1-v17:~$ jobs
[1]+  Stopped                 vim ~/file_task3.txt
[2]-  Running                 sleep 600 &
user1@eltex-practice2-pg1-v17:~$ fg %1
vim ~/file_task3.txt
user1@eltex-practice2-pg1-v17:~$ jobs
[2]+  Running                 sleep 600 &
```

*Скрин 16) Активация задания **vim** и выход из редактора.*

12) Отправить сигнал *SIGTERM* заданию *sleep 600* и выполнить команду *jobs*

```
user1@eltex-practice2-pg1-v17:~$ jobs
[1]+  Running                 sleep 600 &
user1@eltex-practice2-pg1-v17:~$ kill -15 %1
user1@eltex-practice2-pg1-v17:~$ jobs
[1]+  Terminated              sleep 600
user1@eltex-practice2-pg1-v17:~$
```

*Скрин 17) Завершение процесса сигналом **SIGTERM**.*

13) Создать перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (использовать встроенную команду trap) и отправить сигналы самому себе

```
user1@eltex-practice2-pg1-v17:~$ trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
user1@eltex-practice2-pg1-v17:~$ kill -SIGINT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-pg1-v17:~$ kill -SIGQUIT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-pg1-v17:~$
```

*Скрин 18) Установка перехватчика сигналов **SIGINT** и **SIGQUIT**. Отправка сигналов bash.*

14), 15) Создать несколько зомби-процессов несколько раз выполнив команду: *(sleep 1 & exec /bin/sleep 30) &*
Найти при помощи команды *ps* все зомби-процессы, вывести их *PID* и *PPID*. Попробовать досрочно завершить выполнение зомби-процессов.

```
-bash: syntax error near unexpected token `;'
user1@eltex-practice2-pg1-v17:~$ (sleep 1 & exec /bin/sleep 120) &
[1] 8032
user1@eltex-practice2-pg1-v17:~$ (sleep 1 & exec /bin/sleep 120) &
[2] 8034
user1@eltex-practice2-pg1-v17:~$ (sleep 1 & exec /bin/sleep 120) &
[3] 8036
user1@eltex-practice2-pg1-v17:~$ ps -eo pid,ppid,state,comm | awk 'NR == 1 || $3 == "Z"'
    PID    PPID S COMMAND
   8033    8032 Z sleep
   8035    8034 Z sleep
   8037    8036 Z sleep
user1@eltex-practice2-pg1-v17:~$ kill -SIGKILL 8033
user1@eltex-practice2-pg1-v17:~$ ps -p 8033 -o pid,ppid,state,comm
    PID    PPID S COMMAND
   8033    8032 Z sleep
user1@eltex-practice2-pg1-v17:~$
```

*Скрин 19) Создание зомби-процессов, получение их PID и **PPID**, попытка завершить их досрочно.*

В некоторых случаях процессам нужно получить статус завершения дочернего процесса, к примеру, через системный вызов ***wait()*.** Однако если дочерний процесс уже завершился, но ***wait()*** еще не вызван, дочерний процесс переходит в состояние ***zombie***. В этот момент он уже не занимает память и процессорное время, но ядро сохраняет некоторую информацию о нем, такую как ***PID***, ***exit status*** и пр. Затем, когда ***parent*** вызывает ***wait()*** и получает статус**,** ядро удаляет всю оставшуюся информацию о зомби-процессе.

В нашем случае, когда мы вводим команду ***(sleep 1 & exec /bin/sleep 30) &,*** происходит следующее:

1. поскольку в конце стоит ***&***, то что в скобках уходит в дочерний процесс (***fork()***);
2. далее в этом дочернем процессе вызывается ***sleep 1 &,*** что также переводится в дочерний процесс, но уже существующего дочернего процесса созданного шагом выше.
3. далее вызывается ***exec /bin/sleep 30***. Поскольку он вызывается в первом дочернем процессе из шага 1, он заменяет собой (***sleep***) этот процесс, в результате чего получается такое дерево: **bash** -> **sleep 30** -> **sleep 1.** Иными словами **sleep 30** становится родителем **sleep 1**, т.к. ***exec*** заменяет адресное пространство процесса (***code, data, heap, stack***), но сохраняет ***PID***.
4. поскольку ***sleep*** не ждет завершения дочерних процессов, в отличие от **bash**, **sleep 30** не будет ждать **sleep 1,** и возникает ситуация после завершения **sleep 1**, когда дочерний процесс завершился, но ***wait()*** еще не вызван. Процессу **sleep 1** присваивается статус ***zombie.***
5. по завершению **sleep 30**, **bash** читает его статус, ядро удаляет этот процесс. **sleep 1** становится сиротой и его усыновляет **init** (**PID 1**), или другой процесс приёмник, который получает его статус и освобождает оставшиеся ресурсы.

Сигнал ***SIGKILL*** зомби-процессу не сработал, т.к. процесс уже завершен.